Big Data Processing

Week 8 – problem solving

Apache Mahout, Solr, Storm

RMIT UNIVERSITY

Q2 - Mahout recommender engine discussed in this course is based on the similarity measure between users. What is the logic behind it?

- Mahout is highly efficient in user similarity calculation.
- The users having similar ratings on the same products imply that these users have similar preferences.
- User similarity can be used to select the neighbors.
- The similarity can be explored to cluster users into group so as to identify the right products for a specific user.

Apache Hadoop Ecosystem



Apache Mahout

Apache Mahout is an open-source project that is primarily used in producing scalable machine-learning algorithms

Apache Mahout

Apache Mahout is an open-source project that is primarily used in producing scalable machine-learning algorithms

Machine learning - a branch of artificial intelligence based on the idea that systems can learn from data, identify patterns and make decisions with minimal human intervention.

Apache Mahout

Apache Mahout is an open source project that is primarily used in producing scalable machine learning algorithms

Machine learning - a branch of artificial intelligence based on the idea that systems can learn from data, identify patterns and make decisions with minimal human intervention.

- Supervised learning - classification
- Unsupervised learning - clustering
- Reinforcement learning

Apache Mahout

Apache Mahout is an open source project that is primarily used in producing scalable machine learning algorithms

Machine learning - a branch of artificial intelligence based on the idea that systems can learn from data, identify patterns and make decisions with minimal human intervention.

- Supervised learning - classification
- Unsupervised learning - clustering
- Reinforcement learning
- Feature learning
- Sparse dictionary learning
- Anomaly detection
- Association rules
- …

Apache Mahout

Apache Mahout is an open-source project that is primarily used in producing scalable machine-learning algorithms



A *mahout* is one who drives an elephant as its master. The name comes from its close association with Apache Hadoop which uses an elephant as its logo.

Applications of Mahout

- Foursquare uses the recommender engine of Mahout.

Applications of Mahout

- Foursquare uses the recommender engine of Mahout.
- Twitter uses Mahout for user interest modelling.

Applications of Mahout

- Foursquare uses the recommender engine of Mahout.
- Twitter uses Mahout for user interest modelling.
- Yahoo! uses Mahout for pattern mining.

Apache Mahout

Mahout offers a ready-to-use framework for doing data mining tasks on large volumes of data.

Apache Mahout

Mahout offers a ready-to-use framework for doing data mining tasks on large volumes of data.

Apache Mahout

Mahout offers a ready-to-use framework for doing data mining tasks on large volumes of data.

The algorithms of Mahout are written on top of Hadoop

Apache Mahout

Mahout offers a ready-to-use framework for doing data mining tasks on large volumes of data.

- Includes several MapReduce enabled clustering implementations

Apache Mahout

Mahout offers a ready-to-use framework for doing data mining tasks on large volumes of data.

- Includes several MapReduce enabled clustering implementations
  - such as k-means, fuzzy k-means, Canopy, Dirichlet, and Mean-Shift.

Apache Mahout

Mahout offers a ready-to-use framework for doing data mining tasks on large volumes of data.

- Includes several MapReduce enabled clustering implementations
  - such as k-means, fuzzy k-means, Canopy, Dirichlet, and Mean-Shift.
- Supports Distributed Naive Bayes and Complementary Naive Bayes classification

Apache Mahout

Mahout offers a ready-to-use framework for doing data mining tasks on large volumes of data.

- Includes several MapReduce enabled clustering implementations
  - such as k-means, fuzzy k-means, Canopy, Dirichlet, and Mean-Shift.
- Supports Distributed Naive Bayes and Complementary Naive Bayes classification
- Comes with distributed fitness function capabilities for evolutionary programming.

Apache Mahout

Mahout offers a ready-to-use framework for doing data mining tasks on large volumes of data.

- Includes several MapReduce enabled clustering implementations
  - such as k-means, fuzzy k-means, Canopy, Dirichlet, and Mean-Shift.
- Supports Distributed Naive Bayes and Complementary Naive Bayes classification
- Comes with distributed fitness function capabilities for evolutionary programming.
- Includes matrix and vector libraries.

Apache Mahout

Mahout offers a ready-to-use framework for doing data mining tasks on large volumes of data.

- Includes several MapReduce enabled clustering implementations
  - such as k-means, fuzzy k-means, Canopy, Dirichlet, and Mean-Shift.
- Supports Distributed Naive Bayes and Complementary Naive Bayes classification
- Comes with distributed fitness function capabilities for evolutionary programming.
- Includes matrix and vector libraries.

An example
Mahout Recommender Engine

Mahout Recommender Engine

Recommendation Problem

- **Input**
  - A group of users (e.g., #1, #2, #3, #4)

user	product	rate
1	0	1
1	1	2
1	2	5
1	3	5
1	4	5
2	0	1
2	1	2
2	5	5
2	6	4.5
2	2	5
3	1	2.5
3	2	5
3	3	4
3	4	3
4	0	5
4	1	5
4	2	5
4	3	0

Mahout Recommender Engine

Recommendation Problem

- **Input**
  - A group of users (e.g., #1, #2, #3, #4)

user	product	rate
1	0	1
1	1	2
1	2	5
1	3	5
1	4	5
2	0	1
2	1	2
2	5	5
2	6	4.5
2	2	5
3	1	2.5
3	2	5
3	3	4
3	4	3
4	0	5
4	1	5
4	2	5
4	3	0

Mahout Recommender Engine

Recommendation Problem

- **Input**
  - A group of users (e.g., #1, #2, #3, #4)
  - A set of products, like movies, books, smartphone model, hotels, etc. (e.g., #1,….#100)

user	product	rate
1	0	1
1	1	2
1	2	5
1	3	5
1	4	5
2	0	1
2	1	2
2	5	5
2	6	4.5
2	2	5
3	1	2.5
3	2	5
3	3	4
3	4	3
4	0	5
4	1	5
4	2	5
4	3	0

Mahout Recommender Engine

Recommendation Problem

- **Input**
  - A group of users (e.g., #1, #2, #3, #4)
  - A set of products, (e.g., #1,….#100)
  - The ratings given by users to products

user	product	rate
1	0	1
1	1	2
1	2	5
1	3	5
1	4	5
2	0	1
2	1	2
2	5	5
2	6	4.5
2	2	5
3	1	2.5
3	2	5
3	3	4
3	4	3
4	0	5
4	1	5
4	2	5
4	3	0

Mahout Recommender Engine

Recommendation Problem

- **Input**
  - A group of users (e.g., #1, #2, #3, #4)
  - A set of products, (e.g., #1,….#100)
  - The ratings given by users to products
- **Output**

Given a user (e.g., user #2), recommend products the user did not rate yet but may give high ratings.

user	product	rate
1	0	1
1	1	2
1	2	5
1	3	5
1	4	5
2	0	1
2	1	2
2	5	5
2	6	4.5
2	2	5
3	1	2.5
3	2	5
3	3	4
3	4	3
4	0	5
4	1	5
4	2	5
4	3	0

Mahout Recommender Engine

Recommendation Problem

- **Input**
  - A group of users (e.g., #1, #2, #3, #4)
  - A set of products, (e.g., #1,….#100)
  - The ratings given by users to products
- **Output**

Given a user (e.g., user #2), recommend products the user did not rate yet but may give high ratings (i.e., the user is very likely like the recommended products).



User #2 like Product 3 ?
User #2 like Product 4 ?

user	product	rate
1	0	1
1	1	2
1	2	5
1	3	5
1	4	5
2	0	1
2	1	2
2	5	5
2	6	4.5
2	2	5
3	1	2.5
3	2	5
3	3	4
3	4	3
4	0	5
4	1	5
4	2	5
4	3	0

RMIT Classification: Trusted

Mahout Recommender Engine

```
import java.io.File;
import java.util.List;
import org.apache.mahout.cf.taste.impl.model.file.FileDataModel;
import org.apache.mahout.cf.taste.impl.neighborhood.ThresholdUserNeighborhood;
import org.apache.mahout.cf.taste.impl.recommender.GenericUserBasedRecommender;
import org.apache.mahout.cf.taste.impl.similarity.PearsonCorrelationSimilarity;
import org.apache.mahout.cf.taste.model.DataModel;
import org.apache.mahout.cf.taste.neighborhood.UserNeighborhood;
import org.apache.mahout.cf.taste.recommender.RecommendedItem;
import org.apache.mahout.cf.taste.recommender.UserBasedRecommender;
import org.apache.mahout.cf.taste.similarity.UserSimilarity;

public class Recommender {
   public static void main(String args[]){
      try{
         DataModel datamodel = new FileDataModel(new File("data"));
         UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
         UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                             usersimilarity, datamodel);
         UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                             userneighborhood, usersimilarity);
         List<RecommendedItem> recommendations = recommender.recommend(2, 3);
         for (RecommendedItem recommendation : recommendations) {
                  System.out.println(recommendation);
         }
      }catch(Exception e){}
   }
}
```

RMIT Classification: Trusted

Mahout Recommender Engine

```
import java.io.File;
import java.util.List;
import org.apache.mahout.cf.taste.impl.model.file.FileDataModel;
import org.apache.mahout.cf.taste.impl.neighborhood.ThresholdUserNeighborhood;
import org.apache.mahout.cf.taste.impl.recommender.GenericUserBasedRecommender;
import org.apache.mahout.cf.taste.impl.similarity.PearsonCorrelationSimilarity;
import org.apache.mahout.cf.taste.model.DataModel;
import org.apache.mahout.cf.taste.neighborhood.UserNeighborhood;
import org.apache.mahout.cf.taste.recommender.RecommendedItem;
import org.apache.mahout.cf.taste.recommender.UserBasedRecommender;
import org.apache.mahout.cf.taste.similarity.UserSimilarity;
```

```
public class Recommender {
   public static void main(String args[]){
      try{
         DataModel datamodel = new FileDataModel(new File("data"));
         UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
         UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                             usersimilarity, datamodel);
         UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                             userneighborhood, usersimilarity);
         List<RecommendedItem> recommendations = recommender.recommend(2, 3);
         for (RecommendedItem recommendation : recommendations) {
                  System.out.println(recommendation);
         }
      }catch(Exception e){}
   }
}
```

Mahout java library

RMIT Classification: Trusted

Mahout Recommender Engine

```
import java.io.File;
import java.util.List;
import org.apache.mahout.cf.taste.impl.model.file.FileDataModel;
import org.apache.mahout.cf.taste.impl.neighborhood.ThresholdUserNeighborhood;
import org.apache.mahout.cf.taste.impl.recommender.GenericUserBasedRecommender;
import org.apache.mahout.cf.taste.impl.similarity.PearsonCorrelationSimilarity;
import org.apache.mahout.cf.taste.model.DataModel;
import org.apache.mahout.cf.taste.neighborhood.UserNeighborhood;
import org.apache.mahout.cf.taste.recommender.RecommendedItem;
import org.apache.mahout.cf.taste.recommender.UserBasedRecommender;
import org.apache.mahout.cf.taste.similarity.UserSimilarity;

public class Recommender {
   public static void main(String args[]){
      try{
         DataModel datamodel = new FileDataModel(new File("data"));
         UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
         UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                             usersimilarity, datamodel);
         UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                             userneighborhood, usersimilarity);
         List<RecommendedItem> recommendations = recommender.recommend(2, 3);
         for (RecommendedItem recommendation : recommendations) {
                  System.out.println(recommendation);
         }
      }catch(Exception e){}
   }
}
```

RMIT Classification: Trusted

Mahout Recommender Engine

```
import java.io.File;
import java.util.List;
import org.apache.mahout.cf.taste.impl.model.file.FileDataModel;
import org.apache.mahout.cf.taste.impl.neighborhood.ThresholdUserNeighborhood;
import org.apache.mahout.cf.taste.impl.recommender.GenericUserBasedRecommender;
```

Step1: Create DataModel Object

The DataModel object holds a file that contains the Users, Items, and Preferences details of a product.

```
public class Recommender {
   public static void main(String args[]){
      try{
         DataModel datamodel = new FileDataModel(new File("data"));
         UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
         UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                             usersimilarity, datamodel);
         UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                             userneighborhood, usersimilarity);
         List<RecommendedItem> recommendations = recommender.recommend(2, 3);
         for (RecommendedItem recommendation : recommendations) {
                  System.out.println(recommendation);
         }
      }catch(Exception e){}
   }
}
```

RMIT Classification: Trusted

Mahout Recommender Engine

user	product	rate
1	0	1
1	1	2
1	2	5
1	3	5
1	4	5
2	0	1
2	1	2
2	5	5
2	6	4.5
2	2	5
3	1	2.5
3	2	5
3	3	4
3	4	3
4	0	5
4	1	5
4	2	5
4	3	0

```
import java.io.File;
import java.util.List;
import org.apache.mahout.cf.taste.impl.model.file.FileDataModel;
import org.apache.mahout.cf.taste.impl.neighborhood.ThresholdUserNeighborhood;
import org.apache.mahout.cf.taste.impl.recommender.GenericUserBasedRecommender;
```

Step1: Create DataModel Object

The DataModel object holds a file that contains the Users, Items, and Preferences details of a product.

```
public class Recommender {
   public static void main(String args[]){
      try{
         DataModel datamodel = new FileDataModel(new File("data"));
         UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
         UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                             usersimilarity, datamodel);
         UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                             userneighborhood, usersimilarity);
         List<RecommendedItem> recommendations = recommender.recommend(2, 3);
         for (RecommendedItem recommendation : recommendations) {
                  System.out.println(recommendation);
         }
      }catch(Exception e){}
   }
}
```

RMIT Classification: Trusted

Mahout Recommender Engine

```
import java.io.File;
import java.util.List;
import org.apache.mahout.cf.taste.impl.model.file.FileDataModel;
import org.apache.mahout.cf.taste.impl.neighborhood.ThresholdUserNeighborhood;
import org.apache.mahout.cf.taste.impl.recommender.GenericUserBasedRecommender;
import org.apache.mahout.cf.taste.impl.similarity.PearsonCorrelationSimilarity;
import org.apache.mahout.cf.taste.model.DataModel;
import org.apache.mahout.cf.taste.neighborhood.UserNeighborhood;
import org.apache.mahout.cf.taste.recommender.Recommender;
import org.apache.mahout.cf.taste.recommender.UserBasedRecommender;
import org.apache.mahout.cf.taste.similarity.UserSimilarity;

public class Recommender {
    public static void main(String args[]){
        try{
            DataModel datamodel = new FileDataModel(new File("data"));
            UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
            UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                                usersimilarity, datamodel);
            UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                                userneighborhood, usersimilarity);
            List<RecommendedItem> recommendations = recommender.recommend(2, 3);
            for (RecommendedItem recommendation : recommendations) {
                    System.out.println(recommendation);
            }
        }catch(Exception e){}
    }
}
```

Step2: Create UserSimilarity Object
Create UserSimilarity object
using PearsonCorrelationSimilarity class

Step3: Create UserNeighborhood object

This object computes a "neighborhood" of users like a given user. There are two types of neighborhoods:



```
      try{
         DataModel datamodel = new FileDataModel(new File("data"));
         UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
         UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                             usersimilarity, datamodel);
         UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                             userneighborhood, usersimilarity);
         List<RecommendedItem> recommendations = recommender.recommend(2, 3);
         for (RecommendedItem recommendation : recommendations) {
                  System.out.println(recommendation);
         }
      }catch(Exception e){}
   }
}
```

Step3: Create UserNeighborhood object

This object computes a "neighborhood" of users like a given user. There are two types of neighborhoods:

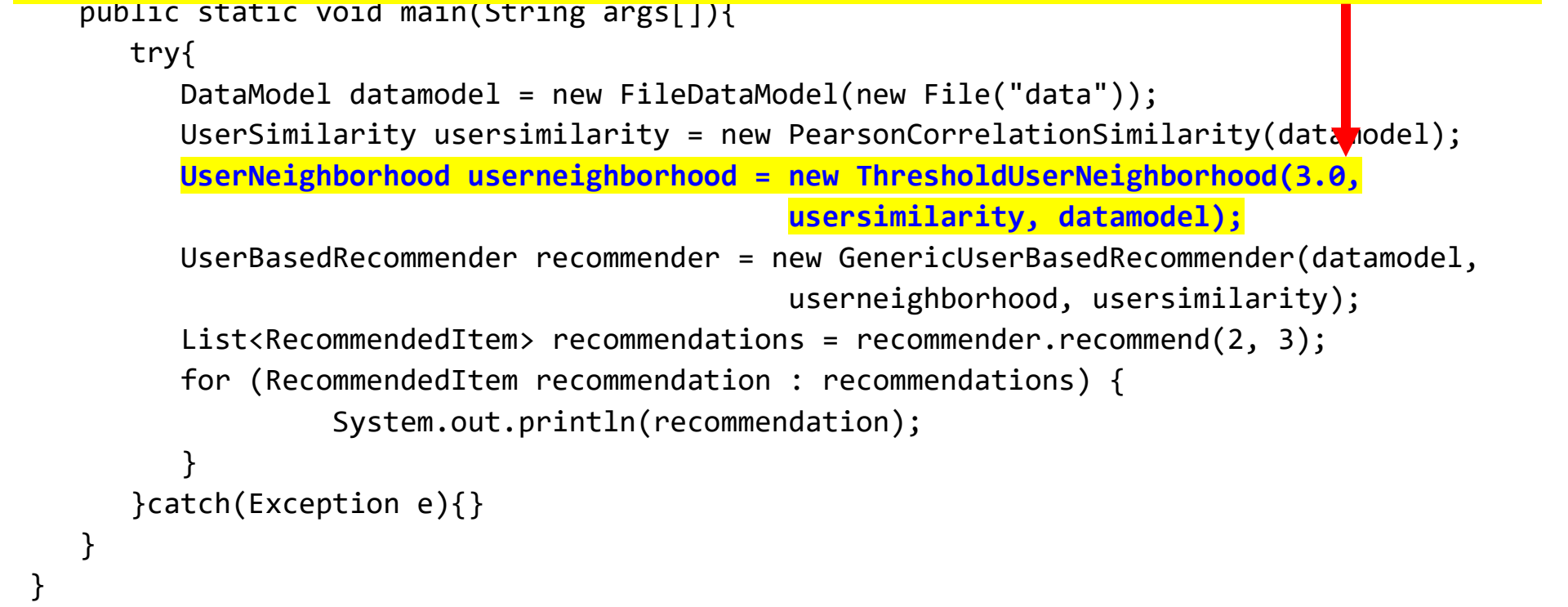
- NearestNUserNeighborhood - This class computes a neighborhood consisting of the nearest *n* users to a given user. "Nearest" is defined by the given UserSimilarity.

```
    try{
        DataModel datamodel = new FileDataModel(new File("data"));
        UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
        UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                            usersimilarity, datamodel);
        UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                            userneighborhood, usersimilarity);
        List<RecommendedItem> recommendations = recommender.recommend(2, 3);
        for (RecommendedItem recommendation : recommendations) {
                System.out.println(recommendation);
        }
    }catch(Exception e){}
  }
}
```

Step3: Create UserNeighborhood object

This object computes a "neighborhood" of users like a given user. There are two types of neighborhoods:

- NearestNUserNeighborhood - This class computes a neighborhood consisting of the nearest *n* users to a given user. "Nearest" is defined by the given UserSimilarity.
- ThresholdUserNeighborhood - This class computes a neighborhood consisting of all the users whose similarity to the given user meets or exceeds a certain threshold. Similarity is defined by the given UserSimilarity.



```
    public static void main(String args[]){
       try{
          DataModel datamodel = new FileDataModel(new File("data"));
          UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
          UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                              usersimilarity, datamodel);
          UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                              userneighborhood, usersimilarity);
          List<RecommendedItem> recommendations = recommender.recommend(2, 3);
          for (RecommendedItem recommendation : recommendations) {
                   System.out.println(recommendation);
          }
       }catch(Exception e){}
    }
}
```

Step3: Create UserNeighborhood object

This object computes a "neighborhood" of users like a given user. There are two types of neighborhoods:

- NearestNUserNeighborhood - This class computes a neighborhood consisting of the nearest *n* users to a given user. "Nearest" is defined by the given UserSimilarity.
- ThresholdUserNeighborhood - This class computes a neighborhood consisting of all the users whose similarity to the given user meets or exceeds a certain threshold. Similarity is defined by the given UserSimilarity.

Here, ThresholdUserNeighborhood is used with the limit of preference to 3.0.

```
        try{
            DataModel datamodel = new FileDataModel(new File("data"));
            UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
            UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                                usersimilarity, datamodel);
            UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                                userneighborhood, usersimilarity);
            List<RecommendedItem> recommendations = recommender.recommend(2, 3);
            for (RecommendedItem recommendation : recommendations) {
                    System.out.println(recommendation);
            }
        }catch(Exception e){}
    }
}
```

RMIT Classification: Trusted

Mahout Recommender Engine

```
import java.io.File;
import java.util.List;
import org.apache.mahout.cf.taste.impl.model.file.FileDataModel;
import org.apache.mahout.cf.taste.impl.neighborhood.ThresholdUserNeighborhood;
import org.apache.mahout.cf.taste.impl.recommender.GenericUserBasedRecommender;
import org.apache.mahout.cf.taste.impl.similarity.PearsonCorrelationSimilarity;
import org.apache.mahout.cf.taste.model.DataModel;
import org.apache.mahout.cf.taste.neighborhood.UserNeighborhood;
import org.apache.mahout.c
import org.apache.mahout.c
import org.apache.mahout.c

public class Reco
   public static void main(String args[]){
      try{
         DataModel datamodel = new FileDataModel(new File("data"));
         UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
         UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                             usersimilarity, datamodel);
         UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                             userneighborhood, usersimilarity);
         List<RecommendedItem> recommendations = recommender.recommend(2, 3);
         for (RecommendedItem recommendation : recommendations) {
                  System.out.println(recommendation);
         }
      }catch(Exception e){}
   }
}
```

Step4: Create Recommender Object

Create UserbasedRecomender object.

RMIT Classification: Trusted

Mahout Recommender Engine

Step5: Recommend Items to a User

Recommend products to a user using the recommend() method of Recommender interface. This method requires two parameters.

```
import java.
import java.
import org.a
import org.a
import org.a
import org.a
import org.a
import org.a
import org.a
import org.a
import org.a

public c
    publi
        tr
            UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
            UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                                usersimilarity, datamodel);
            UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                                userneighborhood, usersimilarity);
            List<RecommendedItem> recommendations = recommender.recommend(2, 3);
            for (RecommendedItem recommendation : recommendations) {
                    System.out.println(recommendation);
            }
        }catch(Exception e){}
    }
}
```

RMIT Classification: Trusted

Mahout Recommender Engine

Step5: Recommend Items to a User

Recommend products to a user using the recommend() method of Recommender interface. This method requires two parameters.

- The first represents the user id of the user to whom we need to send the recommendations, and

```
import java.
import java.
import org.a
import org.a
import org.a
import org.a
import org.a
import org.a
import org.a
import org.a
import org.a

public c
    publi
        tr
            UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
            UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                                usersimilarity, datamodel);
            UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                                userneighborhood, usersimilarity);
            List<RecommendedItem> recommendations = recommender.recommend(2, 3);
            for (RecommendedItem recommendation : recommendations) {
                    System.out.println(recommendation);
            }
        }catch(Exception e){}
    }
}
```

RMIT Classification: Trusted

Mahout Recommender Engine

Step5: Recommend Items to a User

Recommend products to a user using the recommend() method of Recommender interface. This method requires two parameters.

- The first represents the user id of the user to whom we need to send the recommendations, and
- The second represents the number of recommendations to be sent.

```
import java.
import java.
import org.a
import org.a
import org.a
import org.a
import org.a
import org.a
import org.a
import org.a
import org.a

public c
    publi
        tr
            UserSimilarity usersimilarity = new PearsonCorrelationSimilarity(datamodel);
            UserNeighborhood userneighborhood = new ThresholdUserNeighborhood(3.0,
                                                usersimilarity, datamodel);
            UserBasedRecommender recommender = new GenericUserBasedRecommender(datamodel,
                                                userneighborhood, usersimilarity);
            List<RecommendedItem> recommendations = recommender.recommend(2, 3);
            for (RecommendedItem recommendation : recommendations) {
                    System.out.println(recommendation);
            }
        }catch(Exception e){}
    }
}
```

Mahout Recommender Engine

Three products recommended to user #2

```
RecommendedItem[item:12, value:4.8328104]
RecommendedItem[item:13, value:4.6656213]
RecommendedItem[item:14, value:4.331242]
```

Mahout Recommender Engine

```
Three products recommended to user #2

RecommendedItem[item:12, value:4.8328104]
RecommendedItem[item:13, value:4.6656213]
RecommendedItem[item:14, value:4.331242]
```

Is the commendation good or not?

Mahout Recommender Engine

Three products recommended to user #2

```
RecommendedItem[item:12, value:4.8328104]
RecommendedItem[item:13, value:4.6656213]
RecommendedItem[item:14, value:4.331242]
```

user	product	rate
1	0	1
1	1	2
1	2	5
1	3	5
1	4	5
2	0	1
2	1	2
2	5	5
2	6	4.5
2	2	5
3	1	2.5
3	2	5
3	3	4
3	4	3
4	0	5
4	1	5
4	2	5
4	3	0

Is the commendation good or not?

We can use the recommender to

1. predict the values that #2 gives to products (e.g., 0. 1. 5. 6. 2),
2. compare with the true value that #2 gives to the same products.

Mahout Recommender Engine

Three products recommended to user #2

```
RecommendedItem[item:12, value:4.8328104]
RecommendedItem[item:13, value:4.6656213]
RecommendedItem[item:14, value:4.331242]
```

user	product	rate
1	0	1
1	1	2
1	2	5
1	3	5
1	4	5
2	0	1
2	1	2
2	5	5
2	6	4.5
2	2	5
3	1	2.5
3	2	5
3	3	4
3	4	3
4	0	5
4	1	5
4	2	5
4	3	0

Is the commendation good or not?

We can use the recommender to

1. predict the values that #2 gives to products (e.g., 0. 1. 5. 6. 2),
2. compare with the true value that #2 gives to the same products.

Mahout Recommender Engine

Is the commendation good or not?

We can use the recommender to

1. predict the values that #2 gives to products (e.g., 0. 1. 5. 6. 2),
2. compare with the true value that #2 gives to the same products.

```
public class EvaluateRecommender {
    public static void main(String[] args) throws IOException, TasteException {
        // TODO Auto-generated method stub
        DataModel model = new FileDataModel(new File("data/dataset.csv"));
        RecommenderEvaluator evaluator = new AverageAbsoluteDifferenceRecommenderEvaluator();
        RecommenderBuilder builder = new MyRecommenderBuilder();
        double result = evaluator.evaluate(builder, null, model, 0.9, 1.0);
        System.out.println(result);
    }
}

class MyRecommenderBuilder implements RecommenderBuilder {
    public Recommender buildRecommender(DataModel dataModel) throws TasteException {
        UserSimilarity similarity = new PearsonCorrelationSimilarity(dataModel);
        UserNeighborhood neighborhood = new NearestNUserNeighborhood(2, similarity, dataModel);
        return new GenericUserBasedRecommender(dataModel, neighborhood, similarity);
    }
}
```

Mahout Recommender Engine

Is the commendation good or not?

We can use the recommender to

1. predict the values that #2 gives to products (e.g., 0. 1. 5. 6. 2),
2. compare with the true value that #2 gives to the same products.

```
public class EvaluateRecommender {
    public static void main(String[] args) throws IOException, TasteException {
        // TODO Auto-generated method stub
        DataModel model = new FileDataModel(new File("data/dataset.csv"));
        RecommenderEvaluator evaluator = new AverageAbsoluteDifferenceRecommenderEvaluator();
        RecommenderBuilder builder = new MyRecommenderBuilder();
        double result = evaluator.evaluate(builder, null, model, 0.9, 1.0);
        System.out.println(result);
    }
}
```

Recommendation Model

```
class MyRecommenderBuilder implements RecommenderBuilder {
    public Recommender buildRecommender(DataModel dataModel) throws TasteException {
        UserSimilarity similarity = new PearsonCorrelationSimilarity(dataModel);
        UserNeighborhood neighborhood = new NearestNUserNeighborhood(2, similarity, dataModel);
        return new GenericUserBasedRecommender(dataModel, neighborhood, similarity);
    }
}
```

Mahout Recommender Engine

Is the commendation good or not?

We can use the recommender to

1. predict the values that #2 gives to products (e.g., 0. 1. 5. 6. 2),
2. compare with the true value that #2 gives to the same products.

Evaluate Recommendation Model

```
public class EvaluateRecommender {
    public static void main(String[] args) throws IOException, TasteException {
        // TODO Auto-generated method stub
        DataModel model = new FileDataModel(
        RecommenderEvaluator evaluator = new
        RecommenderBuilder builder = new MyRecommenderBuilder();
        double result = evaluator.evaluate(builder, null, model, 0.9, 1.0);
        System.out.println(result);
    }
}
```

Recommendation Model

```
class MyRecommenderBuilder implements RecommenderBuilder {
    public Recommender buildRecommender(DataModel dataModel) throws TasteException {
        UserSimilarity similarity = new PearsonCorrelationSimilarity(dataModel);
        UserNeighborhood neighborhood = new NearestNUserNeighborhood(2, similarity, dataModel);
        return new GenericUserBasedRecommender(dataModel, neighborhood, similarity);
    }
}
```

Mahout Recommender Engine

Is the commendation good or not?

We can use the recommender to

1. predict the values that #2 gives to products (e.g., 0. 1. 5. 6. 2),
2. compare with the true value that #2 gives to the same products.

```
public class EvaluateRecommender {
    public static void main(String[] args) throws IOException, TasteException {
        // TODO Auto-generated method stub
        DataModel model = new FileDataModel(
        RecommenderEvaluator evaluator = new
        RecommenderBuilder builder = new MyRecommenderBuilder();
        double result = evaluator.evaluate(builder, null, model, 0.9, 1.0);
        System.out.println(result);
    }
}
```

Evaluate Recommendation Model

90% training data and 10% test data

```
class MyRecommenderBuilder implements RecommenderBuilder {
    public Recommender buildRecommender(DataModel dataModel) throws TasteException {
        UserSimilarity similarity = new PearsonCorrelationSimilarity(dataModel);
        UserNeighborhood neighborhood = new NearestNUserNeighborhood(2, similarity, dataModel);
        return new GenericUserBasedRecommender(dataModel, neighborhood, similarity);
    }
}
```

Recommendation Model

Q2 - Mahout recommender engine discussed in this course is based on the similarity measure between users. What is the logic behind it?

- Mahout is highly efficient in user similarity calculation.
- The users having the similar ratings on the same products imply that these users have similar preferences.
- User similarity can be used to select the neighbors.
- The similarity can be explored to cluster users into group so as to identify the right products for a specific user.

Q2 - Mahout recommender engine discussed in this course is based on the similarity measure between users. What is the logic behind it?



Mahout is highly effieicent in user similarity calculation.

- The users having the similar ratings on same products imply that these users have the similar preferences.
- User similarity can be used to select the neighbours.
- The similarity can be explored to cluster users into group so as to identify the right products for a specific user.

Q2 - Mahout recommder engine discussed in this course is based on the similarity measure between users. What is the logic behind it?



Mahout is highly effieicent in user similarity calculation.



The users having the similar ratings on same products imply that these users have the similar preferences.

- User similarity can be used to select the neighbours.
- The similarity can be explored to cluster users into group so as to identify the right products for a specific user.

Q2 - Mahout recommder engine discussed in this course is based on the similarity measure between users. What is the logic behind it?


Mahout is highly effieicent in user similarity calculation.


The users having the similar ratings on same products imply that these users have the similar preferences.


User similarity can be used to select the neighbours.

- The similarity can be explored to cluster users into group so as to identify the right products for a specific user.

Q2 - Mahout recommder engine discussed in this course is based on the similarity measure between users. What is the logic behind it?

- ✗ Mahout is highly effieicent in user similarity calculation.
- ✓ The users having the similar ratings on same products imply that these users have the similar preferences.
- ✗ User similarity can be used to select the neighbours.
- ✗ The similarity can be explored to cluster users into group so as to identify the right products for a specific user.

Q3 - Suppose you are a file system manager. If you wan to retrieve files based on the field "keywords" and/or the field "body", what will you do?

- Using solr, create an index on field "keywords" or an index on field "body".
- Using solr, create an index on field "keywords" and an index on field "body" respectively.
- Using mahout to detect the associate rules between the two fields and then retrieve information by exploring the rules.
- Using mahout to create inverted index for the fields repsectively.

Apache Hadoop Ecosystem - Solr



https://lucene.apache.org/solr/guide/8_1/solr-tutorial.html

What is Solr?

Apache Solr is a fast open-source Java search server.

What is Solr?

Apache Solr is a fast open-source Java

search server.

Solr enables you to easily create search engines which searches websites, databases and files.

Basic Solr Concepts

Indexing

- Solr is able to achieve fast search responses because, instead of searching the text directly, it searches an index instead.

Basic Solr Concepts

Indexing

- Solr is able to achieve fast search responses because, instead of searching the text directly, it searches an index instead.
- This type of index is called an **inverted index**,
  - It inverts a page-centric data structure (page->words) to a keyword-centric data structure (word->pages).

Basic Solr Concepts

Indexing

- Solr is able to achieve fast search responses because, instead of searching the text directly, it searches an index instead.
- This type of index is called an **inverted index**,
  - It inverts a page-centric data structure (page->words) to a keyword-centric data structure (word->pages).

Basic Solr Concepts

Indexing

- Solr is able to achieve fast search responses because, instead of searching the text directly, it searches an index instead.
- This type of index is called an **inverted index**,
  - It inverts a page-centric data structure (page->words) to a keyword-centric data structure (word->pages).

p1: “This is a dog”
p2: “The sky is blue”
p3: “The sea is blue”
p4: “The story of a dog and a cat”
p5: “How to have a dog with blue blood?”
p6: “The movie is about a love story”

6 pages (page->words)

Basic Solr Concepts

Indexing

- Solr is able to achieve fast search responses because, instead of searching the text directly, it searches an index instead.
- This type of index is called an **inverted index**,
  - It inverts a page-centric data structure (page->words) to a keyword-centric data structure (word->pages).

p1: “This is a dog”
p2: “The sky is blue”
p3: “The sea is blue”
p4: “The story of a dog and a cat”
p5: “How to have a dog with blue blood?”
p6: “The movie is about a love story”

6 pages (page->words)



dog: p1, p4, p5
cat: p4
sky: p2
sea: p3
blue: p2, p3, p5
story: p5, p6
Movie: p6
love: p6
Blood: p5

inverted index (word->pages)

Basic Solr Concepts

Indexing

- Solr is able to achieve fast search responses because, instead of searching the text directly, it searches an index instead.
- This type of index is called an **inverted index**,
  - It inverts a page-centric data structure (page->words) to a keyword-centric data structure (word->pages).

p1: “This is a dog”
p2: “The sky is blue”
p3: “The sea is blue”
p4: “The story of a dog and a cat”
p5: “How to have a dog with blue blood?”
p6: “The movie is about a love story”

6 pages (page->words)



dog: p1, p4, p5
cat: p4
sky: p2
sea: p3
blue: p2, p3, p5
story: p5, p6
Movie: p6
love: p6
Blood: p5

inverted index (word->pages)

Basic Solr Concepts

Solr stores this index in a directory called **index** in the data directory.

Indexing

- Solr is able to achieve fast search responses because, instead of searching the text directly, it searches an index instead.
- This type of index is called an **inverted index**,
  - It inverts a page-centric data structure (page->words) to a keyword-centric data structure (word->pages).

p1: “This is a dog”
p2: “The sky is blue”
p3: “The sea is blue”
p4: “The story of a dog and a cat”
p5: “How to have a dog with blue blood?”
p6: “The movie is about a love story”

6 pages (page->words)



dog: p1, p4, p5
cat: p4
sky: p2
sea: p3
blue: p2, p3, p5
story: p5, p6
Movie: p6
love: p6
Blood: p5

inverted index (word->pages)

Basic Solr Concepts

Schema

- Before adding documents to Solr, the schema need to be specified in a file called **schema.xml**. The schema declares:

Basic Solr Concepts

Schema

- Before adding documents to Solr, the schema need to be specified in a file called **schema.xml**. The schema declares:
  - what kinds of fields there are
    - Float, long, double, date, text

Basic Solr Concepts

Schema

- Before adding documents to Solr, the schema need to be specified in a file called **schema.xml**. The schema declares:
  - what kinds of fields there are
    - Float, long, double, date, text
  - which field should be used as the unique/primary key

Basic Solr Concepts

Schema

- Before adding documents to Solr, the schema need to be specified in a file called **schema.xml**. The schema declares:
  - what kinds of fields there are
    - Float, long, double, date, text
  - which field should be used as the unique/primary key
  - which fields are required

Basic Solr Concepts

Schema

- Before adding documents to Solr, the schema need to be specified in a file called **schema.xml**. The schema declares:
  - what kinds of fields there are
    - Float, long, double, date, text
  - which field should be used as the unique/primary key
  - which fields are required
  - how to index and search each field

Basic Solr Concepts

Schema

- Before adding documents to Solr, the schema need to be specified in a file called **schema.xml**. The schema declares:
  - what kinds of fields there are
    - Float, long, double, date, text
  - which field should be used as the unique/primary key
  - which fields are required
  - how to index and search each field
    - **indexed** fields are fields which undergo an analysis phase, and are added to the index.

Basic Solr Concepts

Schema

- Before adding documents to Solr, the schema need to be specified in a file called **schema.xml**. The schema declares:
  - what kinds of fields there are
    - Float, long, double, date, text
  - which field should be used as the unique/primary key
  - which fields are required
  - how to index and search each field
    - **indexed** fields are fields which undergo an analysis phase, and are added to the index.

Analysis Phase
When data is added to Solr, it goes through a series of transformations

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

Search for word "foo" in the title field.

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

Search for word "foo" in the title field.

Search for phrase "foo bar" in the title field.

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

Search for word "foo" in the title field.

Search for phrase "foo bar" in the title field.

Search for phrase "foo bar" in the title field AND the phrase "quick fox" in the body field.

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

Search for word "foo" in the title field.

Search for phrase "foo bar" in the title field.

Search for phrase "foo bar" in the title field AND the phrase "quick fox" in the body field.

Search for either the phrase "foo bar" in the title field AND the phrase "quick fox" in the body field, or the word "fox" in the title field.

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

Search for word "foo" in the title field.

Search for phrase "foo bar" in the title field.

Search for phrase "foo bar" in the title field AND the phrase "quick fox" in the body field.

Search for either the phrase "foo bar" in the title field AND the phrase "quick fox" in the body field, or the word "fox" in the title field.

Search for word "foo" and not "bar" in the title field.

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

- Wildcard matching

```
title:foo*
title:foo*bar
```

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

- Wildcard matching

```
title:foo*
title:foo*bar
```

Search for any word that starts with "foo" in the title field.

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

- Wildcard matching

```
title:foo*
title:foo*bar
```

Search for any word that starts with "foo" in the title field.

Search for any word that starts with "foo" and ends with bar in the title field.

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

- Wildcard matching

```
title:foo*
title:foo*bar
```

Search for any word that starts with "foo" in the title field.

Search for any word that starts with "foo" and ends with bar in the title field.

Lucene doesn't support using a * symbol as the first character of a search

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

- Wildcard matching

```
title:foo*
title:foo*bar
```

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

- Wildcard matching

```
title:foo*
title:foo*bar
```

- Range searches

```
mod_date:[20020101 TO 20030101]
```

Range Queries can be inclusive or exclusive of the upper and lower bounds. Sorting is done lexicographically.

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

- Wildcard matching

```
title:foo*
title:foo*bar
```

- Range searches

```
mod_date:[20020101 TO 20030101]
```

Range Queries can be inclusive or exclusive of the upper and lower bounds. Sorting is done lexicographically.

Square brackets [&] - including the upper and lower bound.
e.g., [1 To 5] => 1, 2, 3, 4, 5
Curly brackets { & } - excluding the upper and lower bounds themselves.
e.g., {1 To 5] => 2, 3, 4, 5

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

- Wildcard matching

```
title:foo*
title:foo*bar
```

- Range searches

```
mod_date:[20020101 TO 20030101]
```

Solr Query Syntax

Solr is powerd by the standard Lucene query parser

- Keyword matching

```
title:foo
title:"foo bar"
title:"foo bar" AND body:"quick fox"
(title:"foo bar" AND body:"quick fox") OR title:fox
title:foo -title:bar
```

- Wildcard matching

```
title:foo*
title:foo*bar
```

- Range searches

```
mod_date:[20020101 TO 20030101]
```

- Proximity matching

```
“jakarta apache"~10
```

search for "apache" and "jakarta" within 10 words of each other in a document

Solr Query Syntax

- Boosts

Query-time boosts allow one to specify which terms/clauses are "more important". The higher the boost factor, the more relevant the term will be, and therefore the higher the corresponding document scores.

Solr Query Syntax

- Boosts

Query-time boosts allow one to specify which terms/clauses are "more important". The higher the boost factor, the more relevant the term will be, and therefore the higher the corresponding document scores.

A typical boosting technique is assigning higher boosts to title matches than to body content matches:

```
(title:foo OR title:bar)^1.5 (body:foo OR body:bar)
```

Solr Query Syntax

- Boosts

Query-time boosts allow one to specify which terms/clauses are "more important". The higher the boost factor, the more relevant the term will be, and therefore the higher the corresponding document scores.

A typical boosting technique is assigning higher boosts to title matches than to body content matches:

```
(title:foo OR title:bar)^1.5 (body:foo OR body:bar)
```

In addition, Solr also has Solr-specific query syntax which is different from the standard Lucene query syntax

Q3 - Suppose you are a file system manager. If you want to retrieve files based on the field “title" and/or the field "body", what will you do?

- Using solr, create an index on field “title" or an index on field "body".
- Using solr, create an index on field “title" and an index on field "body" respectively.
- Using mahout to detect the associate rules between the two fields and then retrieve information by exploring the rules.
- Using mahout to create inverted index for the fields respectively.

Q3 - Suppose you are a file system manager. If you wan to retrieve files based on the field "keywords" and/or the field "body", what will you do?



Using solr, create an index on field "keywords" or an index on field "body".

- Using solr, create an index on field "keywords" and an index on field "body" respectively.
- Using mahout to detect the associate rules between the two fields and then retrieve information by exploring the rules.
- Using mahout to create inverted index for the fields respectively.

Q3 - Suppose you are a file system manager. If you wan to retrieve files based on the field "keywords" and/or the field "body", what will you do?



Using solr, create an index on field "keywords" or an index on field "body".



Using solr, create an index on field "keywords" and an index on field "body" respectively.

- Using mahout to detect the associate rules between the two fields and then retrieve information by exploring the rules.
- Using mahout to create inverted index for the fields respectively.

Q3 - Suppose you are a file system manager. If you wan to retrieve files based on the field "keywords" and/or the field "body", what will you do?



Using solr, create an index on field "keywords" or an index on field "body".



Using solr, create an index on field "keywords" and an index on field "body" respectively.



Using mahout to detect the associate rules between the two fields and then retrieve information by exploring the rules.

- Using mahout to create inverted index for the fields respectively.

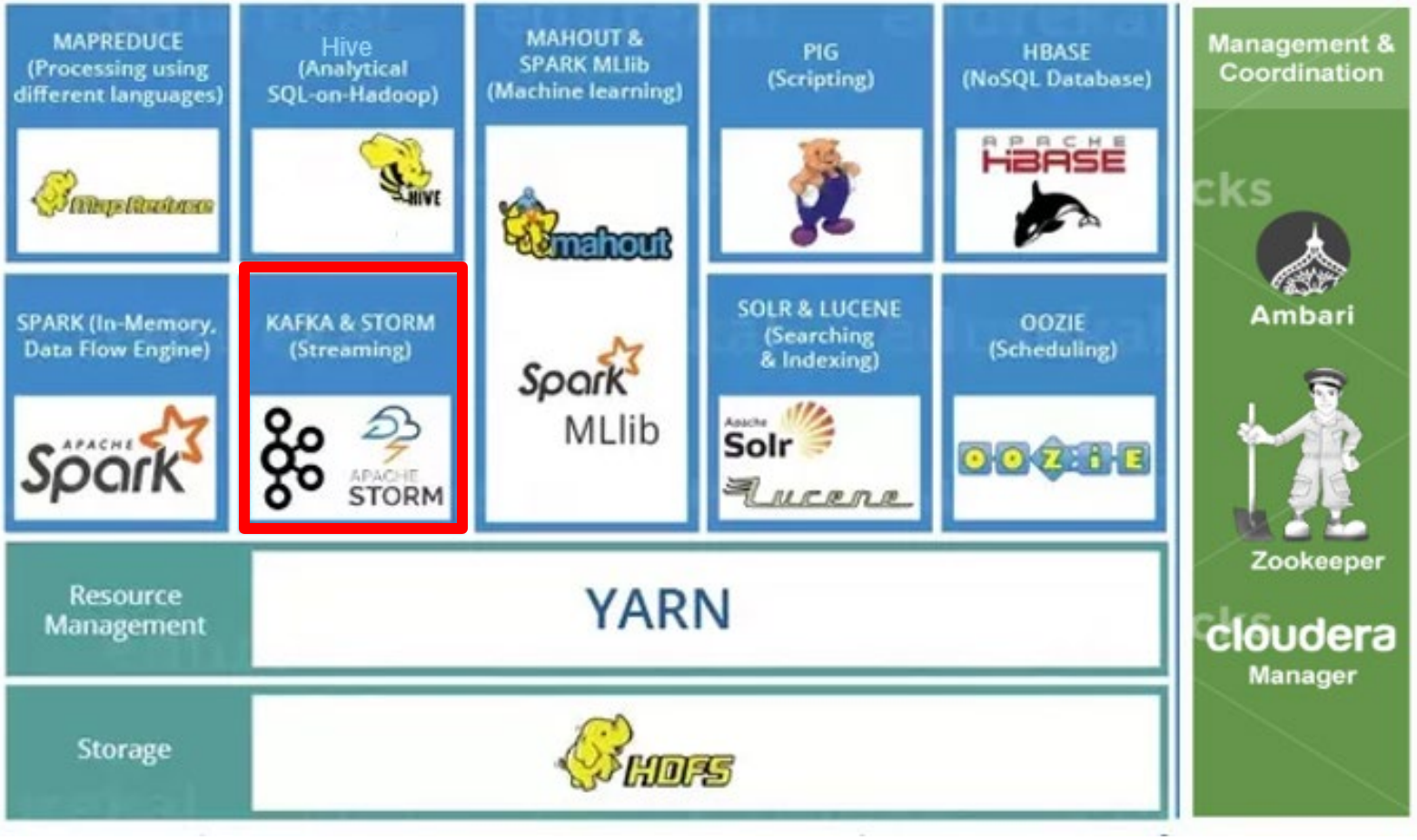
Q3 - Suppose you are a file system manager. If you wan to retrieve files based on the field "keywords" and/or the field "body", what will you do?

- ✗ Using solr, create an index on field "keywords" or an index on field "body".
- ✓ Using solr, create an index on field "keywords" and an index on field "body" respectively.
- ✗ Using mahout to detect the associate rules between the two fields and then retrieve information by exploring the rules.
- ✗ Using mahout to create inverted index for the fields respectively.

Q4 - Select the two correct statements.

- The output stream of a bolt can be the input of another bolt where we can filter out irrelevant information in the first bolt.
- The real-time application logic is specified inside Storm topology. This is the job of data scientists.
- In Solr, even though stream is an unordered squence of tuples, the output of a bolt can be an ordered sequence of tuples if necessary.
- Workers are also known as bolts or spouts in many scenarios.

Apache Hadoop Ecosystem - Storm



What is Apache Storm?

Apache Storm is a free and open source distributed real-time computation system.

Use-Cases of Apache Storm

- **Twitter** − Twitter is using Apache Storm for its range of “Publisher Analytics products”. “Publisher Analytics Products” process each and every tweets and clicks in the Twitter Platform. Apache Storm is deeply integrated with Twitter infrastructure.

Use-Cases of Apache Storm

- **Twitter** − Twitter is using Apache Storm for its range of “Publisher Analytics products”. “Publisher Analytics Products” process each and every tweets and clicks in the Twitter Platform. Apache Storm is deeply integrated with Twitter infrastructure.
- **NaviSite** − NaviSite is using Storm for Event log monitoring/auditing system. Every logs generated in the system will go through the Storm. Storm will check the message against the configured set of regular expression and if there is a match, then that particular message will be saved to the database.

Use-Cases of Apache Storm

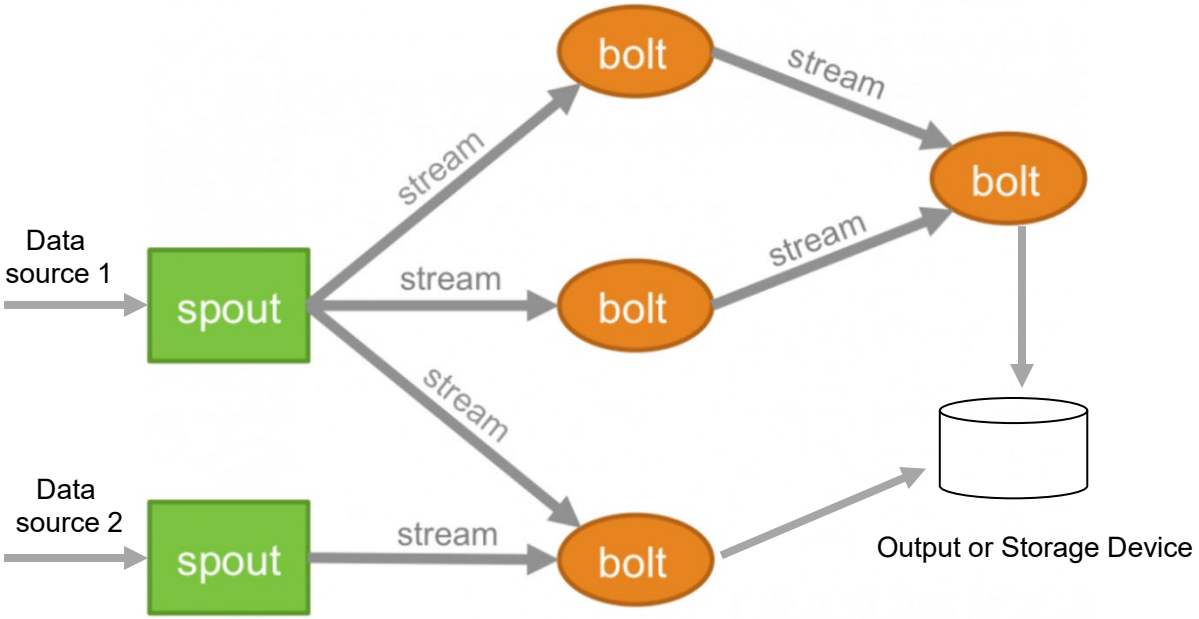
- **Twitter** − Twitter is using Apache Storm for its range of “Publisher Analytics products”. “Publisher Analytics Products” process each and every tweets and clicks in the Twitter Platform. Apache Storm is deeply integrated with Twitter infrastructure.
- **NaviSite** − NaviSite is using Storm for Event log monitoring/auditing system. Every logs generated in the system will go through the Storm. Storm will check the message against the configured set of regular expression and if there is a match, then that particular message will be saved to the database.
- **Wego** − Wego is a travel metasearch engine located in Singapore. Travel related data comes from many sources all over the world with different timing. Storm helps Wego to search real-time data, resolves concurrency issues and find the best match for the end-user.

Apache Storm - Core Concepts

- Apache Storm reads raw stream of real-time data from one end and passes it through a sequence of small processing units and output the processed/useful information at the other end.

Apache Storm - Core Concepts

- Apache Storm reads raw stream of real-time data from one end and passes it through a sequence of small processing units and output the processed/useful information at the other end.

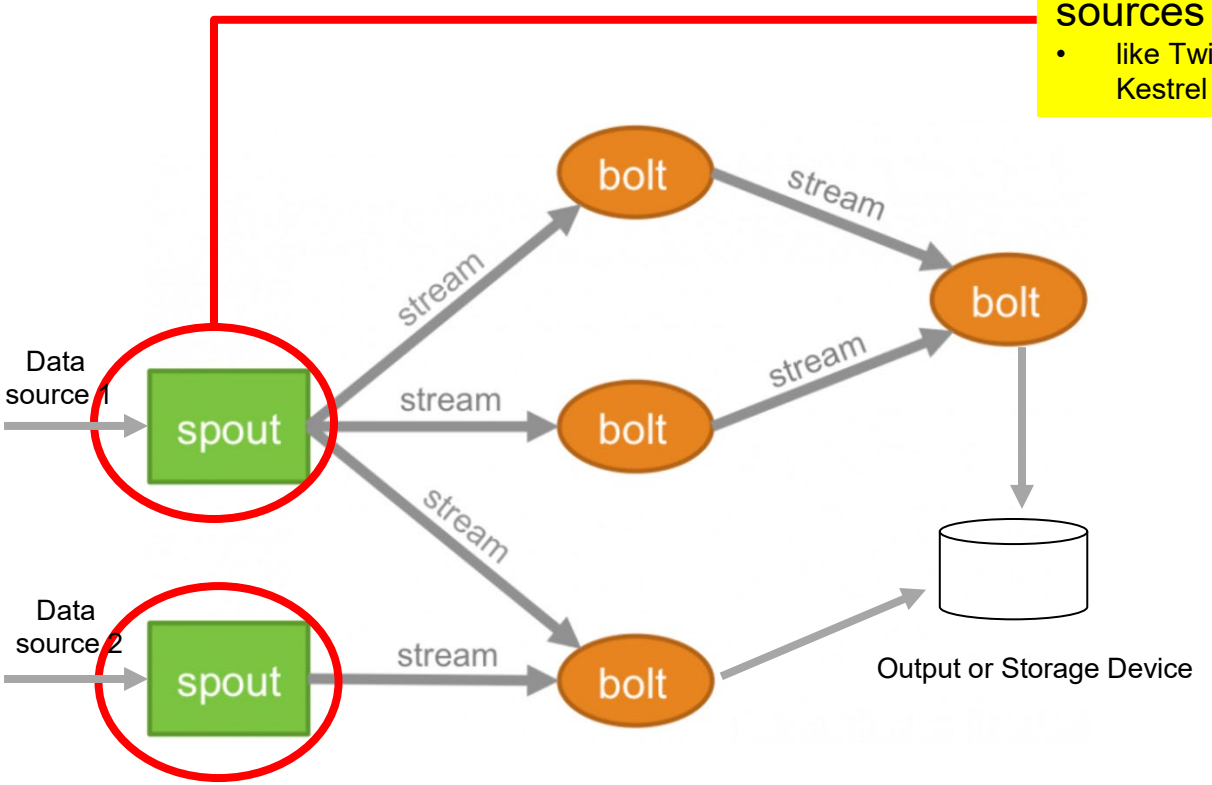


Apache Storm - Core Concepts

- Apache Storm reads raw stream of real-time data from one end and passes it through a sequence of small processing units and output the processed/useful information at the other end.

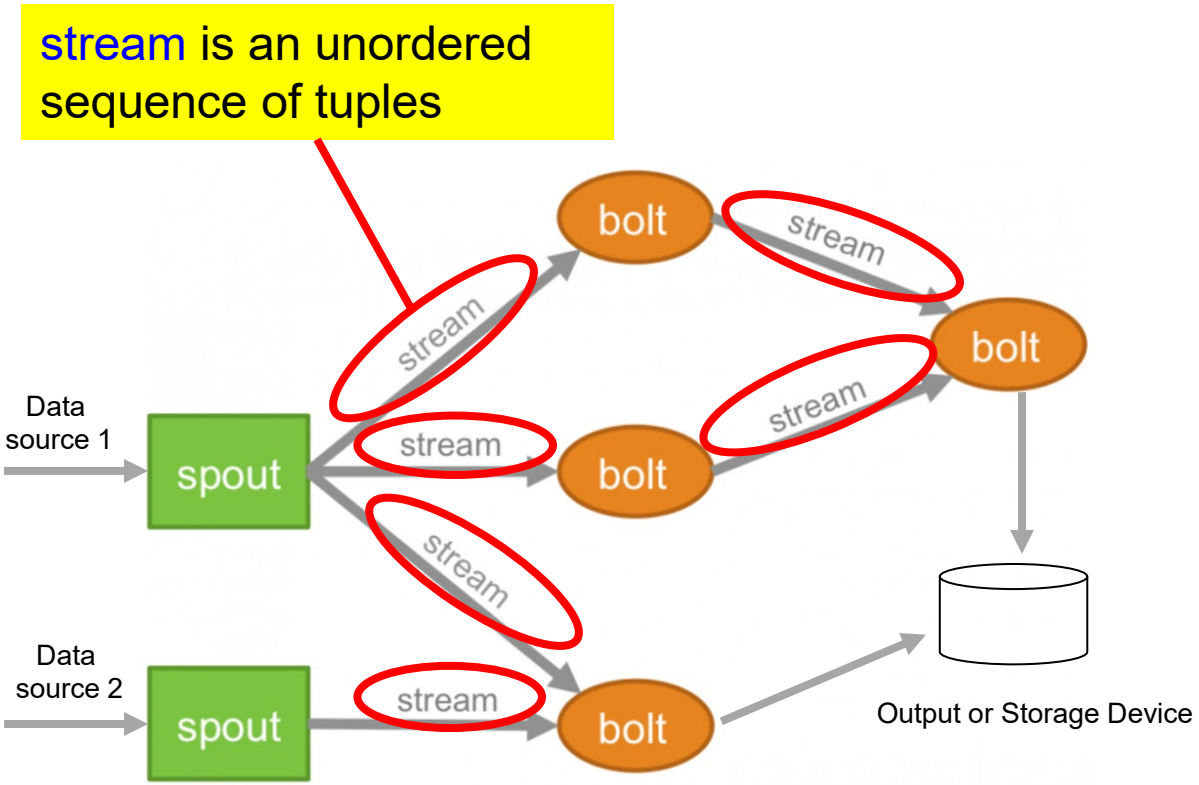
spout accepts input data from raw data sources

- like Twitter Streaming API, Apache Kafka queue, Kestrel queue, etc.



Apache Storm - Core Concepts

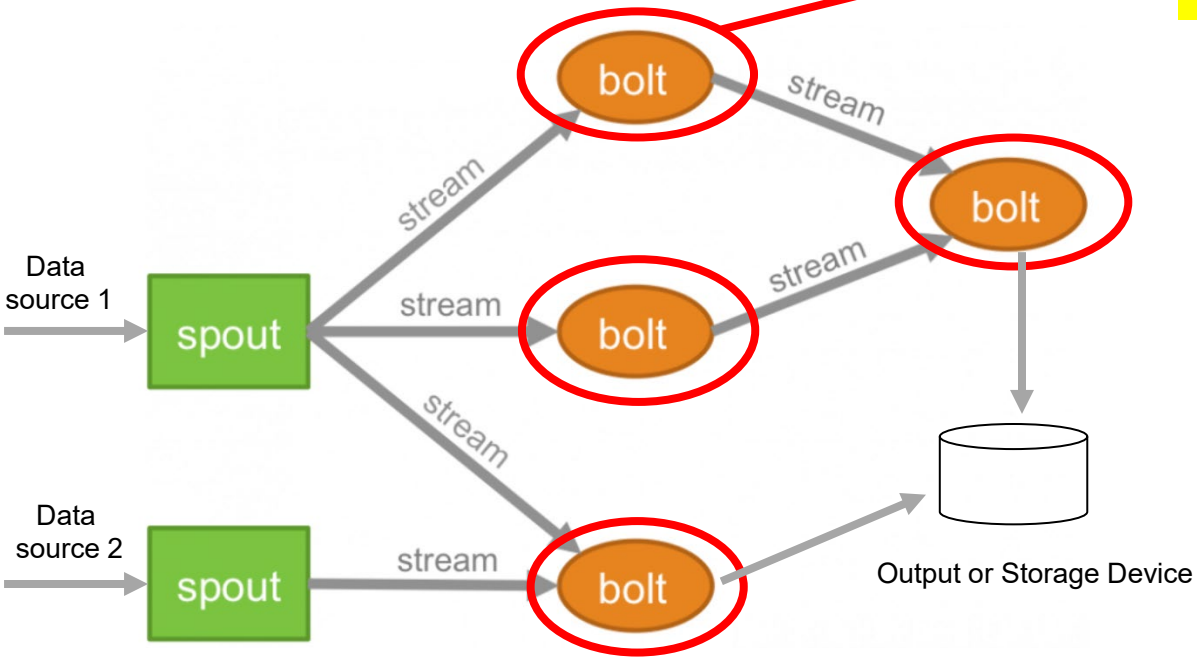
- Apache Storm reads raw stream of real-time data from one end and passes it through a sequence of small processing units and output the processed/useful information at the other end.



Apache Storm - Core Concepts

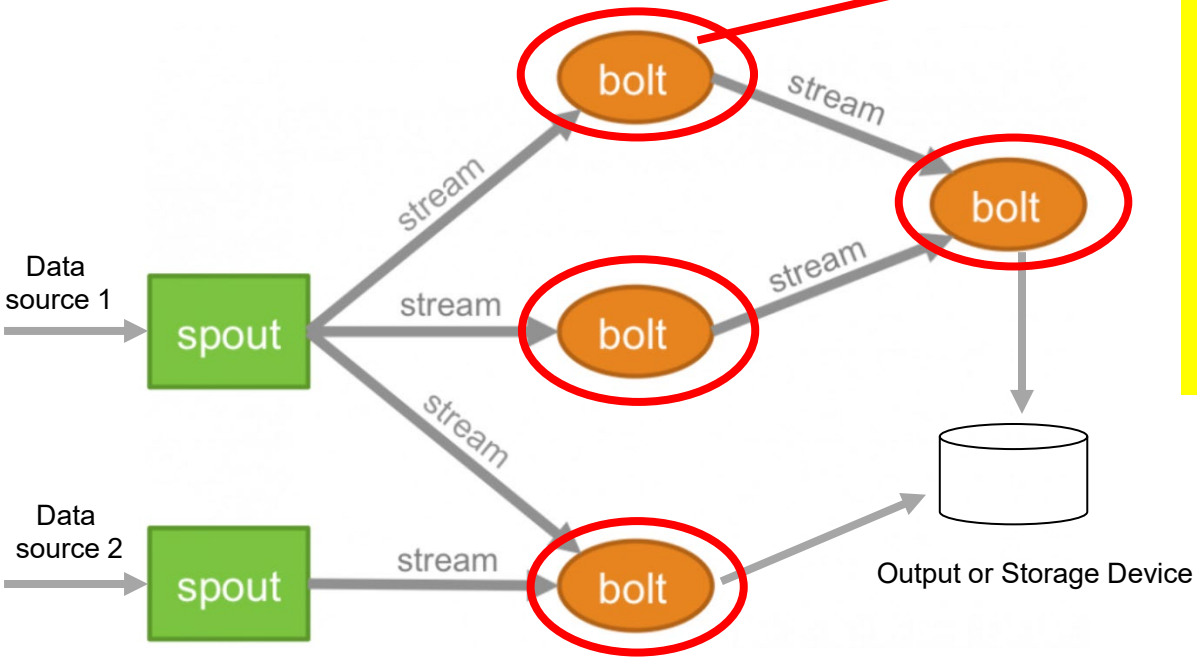
- Apache Storm reads raw stream of real-time data from one end and passes it through a sequence of small processing units and output the processed / useful information at the other end.

bolts are logical processing units.
1. Spouts pass data to bolts
2. bolts process and produce a new output stream



Apache Storm - Core Concepts

- Apache Storm reads raw stream of real-time data from one end and passes it through a sequence of small processing units and output the processed / useful information at the other end.



bolts are logical processing units.

1. Spouts pass data to bolts
2. bolts process and produce a new output stream

bolts can perform the operations of

- filtering,
- aggregation,
- joining,
- interacting with data sources and databases

Apache Storm - Topology

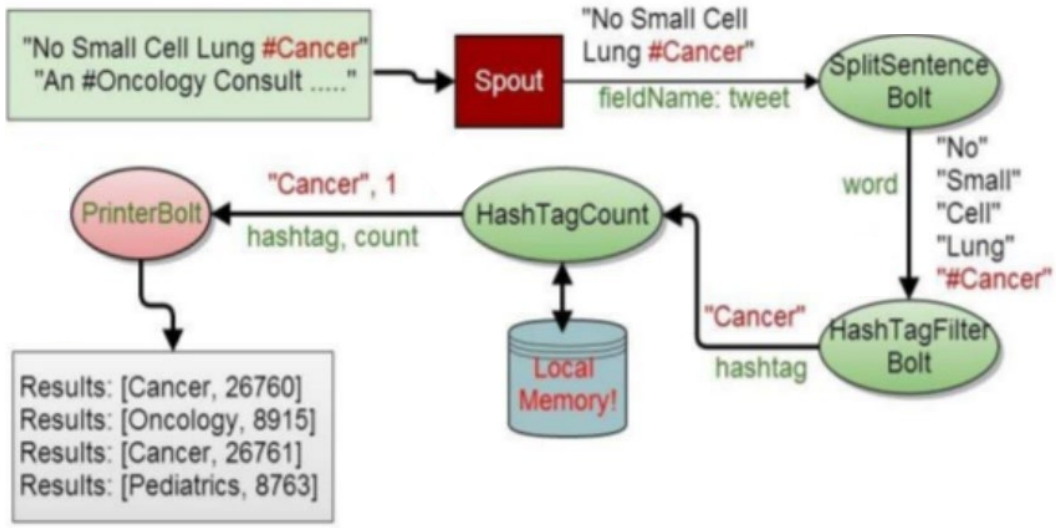
- Spouts and bolts are connected together and they form a topology. Real-time application logic is specified inside Storm topology.
  - In simple words, a topology is a directed graph where vertices are computation and edges are stream of data.

Apache Storm - Topology

- Spouts and bolts are connected together and they form a topology. Real-time application logic is specified inside Storm topology.
  - In simple words, a topology is a directed graph where vertices are computation and edges are stream of data.

Apache Storm - Topology

- Spouts and bolts are connected together and they form a topology. Real-time application logic is specified inside Storm topology.
  - In simple words, a topology is a directed graph where vertices are computation and edges are stream of data.

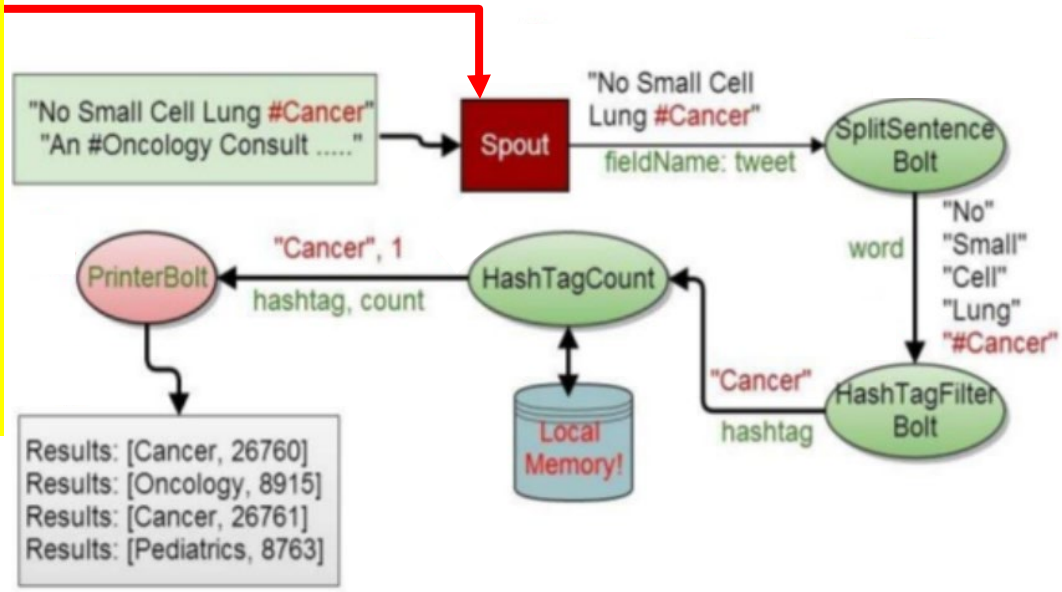


https://www.slideshare.net/edvorkin/storm-intro-v8

Apache Storm - Topology

- Spouts and bolts are connected together and they form a topology. Real-time application logic is specified inside Storm topology.
  - In simple words, a topology is a directed graph where vertices are computation and edges are stream of data.

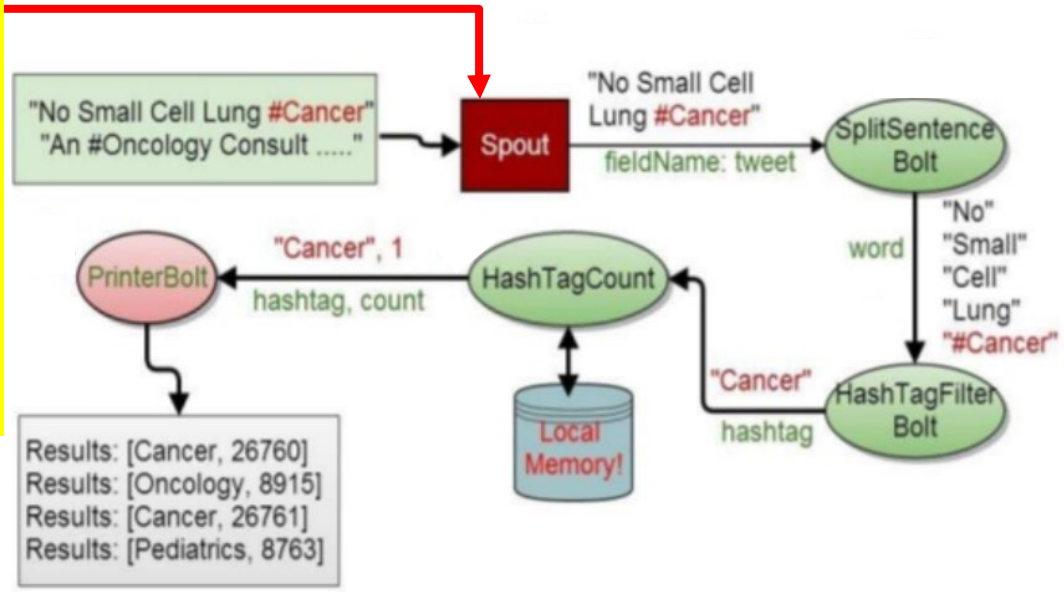
A simple topology starts with spouts.
- Spout emits the data to one or more bolts.
  - Bolt represents a node in the topology having the smallest processing logic
  - The output of a bolt can be emitted into another bolt as input.



Apache Storm - Topology

- Spouts and bolts are connected together and they form a topology. Real-time application logic is specified inside Storm topology.
  - In simple words, a topology is a directed graph where vertices are computation and edges are stream of data.

A simple topology starts with spouts.
- Spout emits the data to one or more bolts.
  - Bolt represents a node in the topology having the smallest processing logic
  - The output of a bolt can be emitted into another bolt as input.

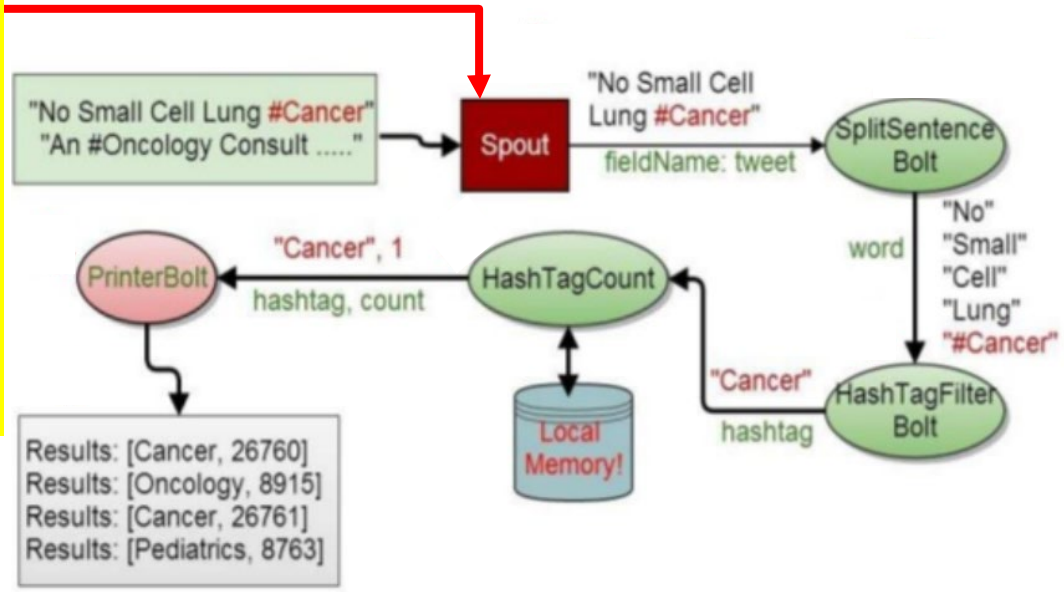


Storm keeps the topology always running, until you kill the topology. Apache Storm's main job is to run the topology and will run any number of topology at a given time.

Apache Storm - Topology

- Spouts and bolts are connected together and they form a topology. Real-time application logic is specified inside Storm topology.
  - In simple words, a topology is a directed graph where vertices are computation and edges are stream of data.

A simple topology starts with spouts.
- Spout emits the data to one or more bolts.
  - Bolt represents a node in the topology having the smallest processing logic
  - The output of a bolt can be emitted into another bolt as input.



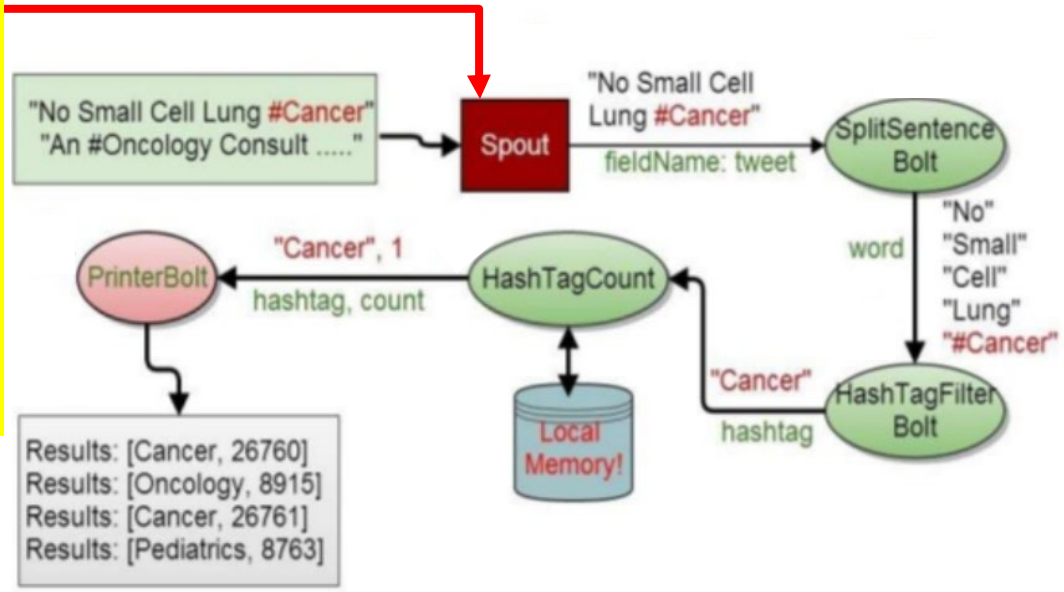
Task
The executions on spout and bolt by Storm is called as “Tasks”.

Storm keeps the topology always running, until you kill the topology. Apache Storm’s main job is to run the topology and will run any number of topology at a given time.

Apache Storm - Topology

- Spouts and bolts are connected together and they form a topology. Real-time application logic is specified inside Storm topology.
  - In simple words, a topology is a directed graph where vertices are computation and edges are stream of data.

- A simple topology starts with spouts.
- Spout emits the data to one or more bolts.
  - Bolt represents a node in the topology having the smallest processing logic
  - The output of a bolt can be emitted into another bolt as input.



Task
The executions on spout and bolt by Storm is called as “Tasks”.

Worker
A topology runs in a distributed manner, on multiple servers of a cluster (worker nodes). A worker may process >=1 tasks

Storm keeps the topology always running, until you kill the topology. Apache Storm’s main job is to run the topology and will run any number of topology at a given time.

RMIT Classification: Trusted

Q4 - Select the two correct statements.

- The output stream of a bolt can be the input of another bolt where we can filter out irrelevant information in the first bolt.
- The real-time application logic is specified inside Storm topology. This is the job of data scientists.
- In Storm, even though stream is an unordered sequence of tuples, the output of a bolt can be an ordered sequence of tuples if necessary.
- Workers are also known as bolts or spouts in many scenarios.

Q4 - Select the two correct statements.

- ✓ The output stream of a bolt can be the input of another bolt where we can filter out irrelevant information in the first bolt.
- The real-time application logic is specified inside Storm topology. This is the job of data scientists.
- In Storm, even though stream is an unordered sequence of tuples, the output of a bolt can be an ordered sequence of tuples if necessary.
- Workers are also known as bolts or spouts in many scenarios.

RMIT Classification: Trusted

Q4 - Select the two correct statements.

- ✓ The output stream of a bolt can be the input of another bolt where we can filter out irrelevant information in the first bolt.
- ✓ The real-time application logic is specified inside Storm topology. This is the job of data scientists.
- In Storm, even though stream is an unordered sequence of tuples, the output of a bolt can be an ordered sequence of tuples if necessary.
- Workers are also known as bolts or spouts in many scenarios.

Q4 - Select the two correct statements.



The output stream of a bolt can be the input of another bolt where we can filter out irrelevant information in the first bolt.



The real-time application logic is specified inside Storm topology. This is the job of data scientists.



In Storm, even though stream is an unordered sequence of tuples, the output of a bolt can be an ordered sequence of tuples if necessary.

- Workers are also known as bolts or spouts in many scenarios.

Q4 - Select the two correct statements.

- ✓ The output stream of a bolt can be the input of another bolt where we can filter out irrelevant information in the first bolt.
- ✓ The real-time application logic is specified inside Storm topology. This is the job of data scientists.
- ✗ In Storm, even though stream is an unordered sequence of tuples, the output of a bolt can be an ordered sequence of tuples if necessary.
- ✗ Workers are also known as bolts or spouts in many scenarios.

Q1

- Apache PIG
- Apache HIVE
- Apache IntelliJ
- Apache Storm
- Apache Solr
- Apache Mahout

1. provides a series of functions that can be organized in a script for carrying out tasks in data flows.
2. can be used for processing structured data and query is SQL-like.
3. may help if you often need to find files with expected information from a large data set in your organization.
4. Needs to specify the way of connecting processing units to enable the continuing information processing in specific applications.
5. makes it possible to build up prediction models by learning from a large volume of training data.
6. saves time and effort of program developers.

Q1

- Apache PIG
- Apache HIVE
- Apache IntelliJ
- Apache Storm
- Apache Solr
- Apache Mahout

1. provides a series of functions that can be organized in a script for carrying out tasks in data flows. Apache PIG
2. can be used for processing structured data and query is SQL-like.
3. may help if you often need to find files with expected information from a large data set in your organization.
4. Needs to specify the way of connecting processing units to enable the continuing information processing in specific applications.
5. makes it possible to build up prediction models by learning from a large volume of training data.
6. saves time and effort of program developers.

Q1

- Apache PIG
- Apache HIVE
- Apache IntelliJ
- Apache Storm
- Apache Solr
- Apache Mahout

1. provides a series of functions that can be organized in a script for carrying out tasks in data flows. Apache PIG
2. can be used for processing structured data and query is SQL-like. Apache HIVE
3. may help if you often need to find files with expected information from a large data set in your organization.
4. Needs to specify the way of connecting processing units to enable the continuing information processing in specific applications.
5. makes it possible to build up prediction models by learning from a large volume of training data.
6. saves time and effort of program developers.

Q1

- Apache PIG
- Apache HIVE
- Apache IntelliJ
- Apache Storm
- Apache Solr
- Apache Mahout

1. provides a series of functions that can be organized in a script for carrying out tasks in data flows. Apache PIG
2. can be used for processing structured data and query is SQL-like. Apache HIVE
3. may help if you often need to find files with expected information from a large data set in your organization. Apache Solr
4. Needs to specify the way of connecting processing units to enable the continuing information processing in specific applications.
5. makes it possible to build up prediction models by learning from a large volume of training data.
6. saves time and effort of program developers.

Q1

- Apache PIG
- Apache HIVE
- Apache IntelliJ
- Apache Storm
- Apache Solr
- Apache Mahout

1. provides a series of functions that can be organized in a script for carrying out tasks in data flows. Apache PIG
2. can be used for processing structured data and query is SQL-like. Apache HIVE
3. may help if you often need to find files with expected information from a large data set in your organization. Apache Solr
4. Needs to specify the way of connecting processing units to enable the continuing information processing in specific applications. Apache Storm
5. makes it possible to build up prediction models by learning from a large volume of training data.
6. saves time and effort of program developers.

Q1

- Apache PIG
- Apache HIVE
- Apache IntelliJ
- Apache Storm
- Apache Solr
- Apache Mahout

1. provides a series of functions that can be organized in a script for carrying out tasks in data flows. Apache PIG
2. can be used for processing structured data and query is SQL-like. Apache HIVE
3. may help if you often need to find files with expected information from a large data set in your organization. Apache Solr
4. Needs to specify the way of connecting processing units to enable the continuing information processing in specific applications. Apache Storm
5. makes it possible to build up prediction models by learning from a large volume of training data. Apache Mahout
6. saves time and effort of program developers.

Q1

- Apache PIG
- Apache HIVE
- Apache IntelliJ
- Apache Storm
- Apache Solr
- Apache Mahout

1. provides a series of functions that can be organized in a script for carrying out tasks in data flows. Apache PIG
2. can be used for processing structured data and query is SQL-like. Apache HIVE
3. may help if you often need to find files with expected information from a large data set in your organization. Apache Solr
4. Needs to specify the way of connecting processing units to enable the continuing information processing in specific applications. Apache Storm
5. makes it possible to build up prediction models by learning from a large volume of training data. Apache Mahout
6. saves time and effort of program developers. Apache IntelliJ

Questions?